

SIMATS ENGINEERING

(Saveetha School of Engineering)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1

Git & Docker Technical Assignment

Project Title: AI InterviewX — An AI-Powered Mock Interview Platform

Course Code	CSA10 / CSA1063
Course Title	Software Engineering
CO Assessed	CO3
Weightage	20%
Total Marks	25
Student Name	Y Praveen Raj
Duration	60 minutes

1. Problem Analysis

1.1 Problem Statement

Technical interview preparation is fragmented, expensive, and lacks personalized, on-demand feedback. Aspiring software engineers rely on generic question banks or costly human mock-interviewers, with no system that adapts questions to a candidate's own resume, role target, and prior answers, and no immediate, structured feedback loop to correct weaknesses before a real interview.

AI InterviewX addresses this gap by combining a Retrieval-Augmented Generation (RAG) pipeline with a conversational interview simulator. The system ingests a candidate's resume and target job description, retrieves relevant technical and behavioral question patterns from a curated knowledge base, and conducts a simulated interview session, followed by AI-generated feedback on correctness, clarity, and confidence.

1.2 Project Objectives

- Build an end-to-end platform that simulates realistic technical and HR interviews using an LLM-driven conversational engine.
- Use RAG to ground interview questions and evaluation feedback in a curated corpus of role-specific interview material rather than relying purely on model memory.
- Provide structured, actionable feedback (strengths, gaps, suggested improvements) after each mock session.
- Package the system so it is reproducible and deployable across environments through containerization.

1.3 Functional Requirements

- Resume and job-description upload and parsing.
- Semantic retrieval of relevant question sets from a vector store (ChromaDB) using sentence-transformer embeddings.
- Conversational interview loop via an LLM (OpenAI API) orchestrated with LangChain.
- REST API layer (FastAPI) exposing session, question, and feedback endpoints.
- Feedback report generation summarizing performance per question and overall.

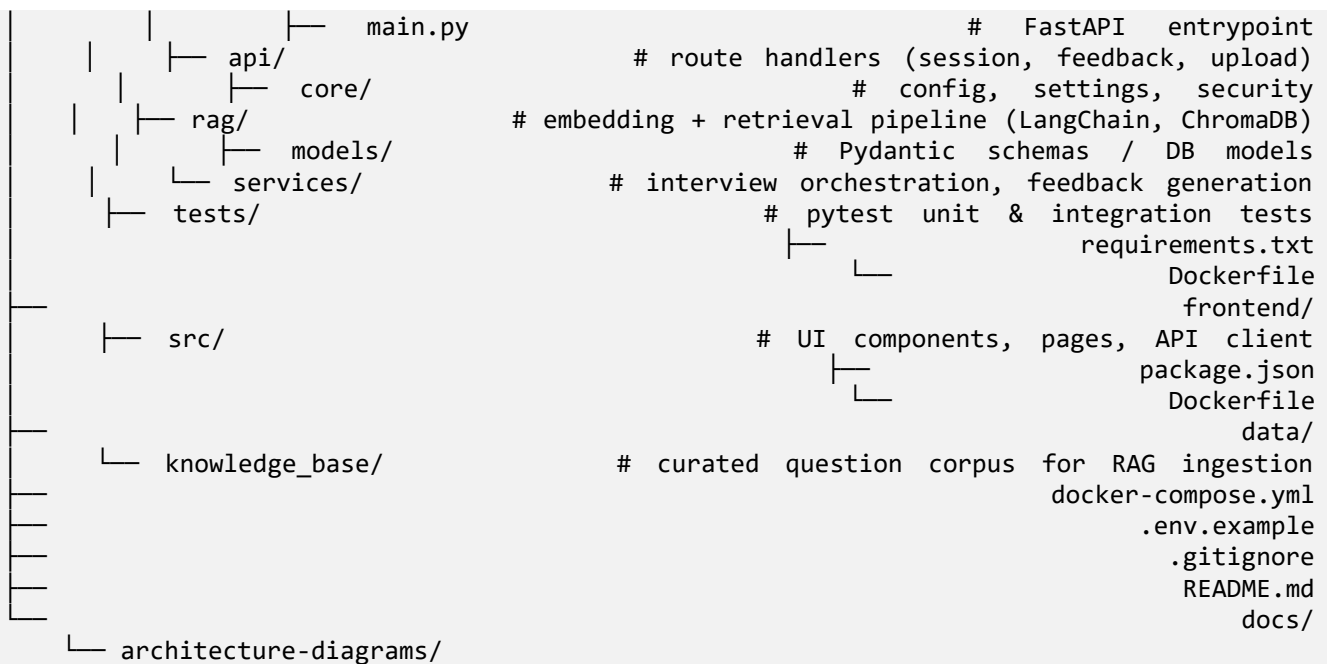
1.4 Expected Outcomes

A working, containerized full-stack application that a candidate can run locally or access via a deployed link, complete an end-to-end mock interview session, and receive a feedback report — all backed by a versioned Git history and a reproducible Docker environment.

2. Project Structure Design

2.1 Repository Folder Structure

```
ai-interviewx/
├──
└── backend/
    └── app/
```



2.2 Justification of Folder and File Purpose

- backend/app/api — isolates HTTP route logic from business logic, keeping endpoints thin and testable.
- backend/app/rag — separates the retrieval pipeline (embeddings, ChromaDB queries) so it can be tuned or swapped without touching orchestration code.
- backend/app/services — houses the interview orchestration and feedback-generation logic; the core domain logic of the product.
- data/knowledge_base — versioned source corpus used to (re)build the vector index, kept outside application code for clarity.
- docker-compose.yml at the root — coordinates backend, frontend, and vector-store services as a single reproducible stack.
- .env.example — documents required environment variables (API keys, DB URLs) without committing secrets.
- README.md and docs/ — onboarding and architecture reference for collaborators.

3. Git Repository Initialization

3.1 Initialization Steps

```

$ git init
$ git add .gitignore README.md
$ git commit -m "chore: initialize repository with README and gitignore"
$ git remote add origin https://github.com/<user>/ai-interviewx.git
$ git branch -M main
$ git push -u origin main
  
```

3.2 Purpose of Essential Git Files

- .git/ — hidden metadata directory holding the full object database, refs, and history; created by git init.
- .gitignore — excludes generated and sensitive files (node_modules/, __pycache__/, .env, chroma_db/) from version control.
- README.md — entry-point documentation describing setup, usage, and architecture for new contributors.
- requirements.txt / package.json — declare reproducible dependency sets for backend and frontend respectively.

4. Git Branching Strategy

4.1 Chosen Model: Simplified Git Flow

A Git Flow-inspired model was adopted with five branch types: main, develop, feature/*, release/*, and hotfix/*.

Branch	Purpose
main	Always reflects production-ready, deployed code.
develop	Integration branch where completed features converge before release.
feature/rag-pipeline, feature/interview-ui	Isolated development of individual features off develop.
release/v1.0	Stabilization and final testing before merging into main.
hotfix/fix-embedding-bug	Urgent production fixes branched directly from main.

4.2 Justification

AI InterviewX has clearly separable modules — RAG pipeline, interview orchestration, and UI — that can be developed in parallel without blocking one another. A Git Flow-style model isolates in-progress feature work in feature branches, keeps develop always integrable, and guarantees main is deployable at any time, which matters for a project that is also containerized and shared as a resume artifact.

5. Version Control Workflow

5.1 Demonstrated Workflow

```

$ git checkout -b feature/rag-pipeline develop
$ git add backend/app/rag/
$ git commit -m "feat(rag): add ChromaDB ingestion and retriever"
$ git checkout -b feature/interview-ui develop
$ git add frontend/src/pages/Interview.jsx
$ git commit -m "feat(ui): add live interview session screen"
$ git checkout develop
$ git merge feature/rag-pipeline
$ git merge feature/interview-ui

```

```
# Merge conflict in backend/app/core/config.py resolved manually
$ git add backend/app/core/config.py
$ git commit -m "merge: resolve config conflict between rag and ui branches"
$ git push origin develop
```

5.2 Purpose of Each Operation

- checkout -b — creates an isolated branch so in-progress work never destabilizes develop.
- add / commit — stages and records a logical, reviewable unit of change.
- merge — integrates completed feature work back into the shared branch.
- conflict resolution — manual reconciliation when two branches edit overlapping lines, ensuring no work is silently lost.
- push — synchronizes local history with the remote so collaborators see the same state.

6. Commit History Analysis

6.1 Sample Commit History

```
a1c9e21 feat(rag): add ChromaDB ingestion and retriever
7f3d8b2 feat(ui): add live interview session screen
e02c5aa merge: resolve config conflict between rag and ui branches
9b41d10 feat(api): add /session/start and /session/answer endpoints
3c88a02 feat(feedback): generate structured post-interview report
d5e7f61 test: add unit tests for retriever and feedback service
b0a34e9 fix(rag): correct embedding dimension mismatch
12fbc44 chore: add Dockerfile for backend service
6d9a012 chore: add docker-compose with backend, frontend, chroma services
aa701de docs: update README with setup and usage instructions
```

6.2 Commit Message Practices

- Conventional Commits prefix (feat, fix, chore, docs, test) makes intent scannable at a glance.
- Each message describes what changed and, briefly, why — never vague messages like "update" or "fix stuff".
- Commits are atomic — one logical change per commit — which keeps git bisect and code review tractable.
- This discipline directly supports project maintenance: a new contributor can reconstruct feature history from log alone.

7. Docker Environment Design

7.1 Why Docker Is Suitable

AI InterviewX depends on a specific combination of Python packages (LangChain, sentence-transformers, ChromaDB client), a Node-based frontend build, and a persistent vector-store service — a dependency footprint that is easy to misconfigure manually. Docker guarantees that the exact same environment runs identically on any evaluator's machine, in CI, or in production, eliminating "works on my machine" failures.

7.2 Components Requiring Containerization

- Backend API service (FastAPI + RAG pipeline) — Python runtime with heavy ML dependencies.
- Frontend service (React build served via Nginx or Node) — isolated Node toolchain.
- Vector database (ChromaDB) — run as its own service with a persistent volume for the index.

8. Dockerfile Development

8.1 Backend Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 Instruction-by-Instruction Explanation

- FROM python:3.11-slim — a minimal base image, keeping the image small while matching the development Python version.
- WORKDIR /app — sets a consistent working directory inside the container for all subsequent instructions.
- COPY requirements.txt . then RUN pip install — copying only the dependency manifest first lets Docker cache this layer, so re-builds after code-only changes skip re-installing packages.
- COPY ./app ./app — brings in application source after dependencies are cached.
- EXPOSE 8000 — documents the port the FastAPI service listens on.
- CMD [...] — the default command that starts the Uvicorn ASGI server when the container runs.

9. Docker Compose Design

9.1 docker-compose.yml

```
version: "3.9"
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    env_file: .env
    depends_on:
      - chromadb
    volumes:
      - ./data/knowledge_base:/app/data/knowledge_base
    networks:
      - interviewx-net
  frontend:
    build: ./frontend
```

```

    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - interviewx-net

chromadb:
  image: chromadb/chroma:latest
  ports:
    - "8001:8000"
  volumes:
    - chroma-data:/chroma/chroma
  networks:
    - interviewx-net

volumes:
  chroma-data:

networks:
  interviewx-net:
    driver: bridge

```

9.2 Services, Networking, Volumes, and Dependencies

- Services — backend, frontend, and chromadb are each defined as independent, horizontally-replaceable services.
- Networking — a dedicated bridge network (interviewx-net) lets services address each other by service name (e.g., backend reaches chromadb at `http://chromadb:8000`) without exposing internal traffic to the host.
- Volumes — chroma-data persists the vector index across container restarts; the `knowledge_base` bind mount lets the corpus be updated without rebuilding the image.
- Environment variables — supplied via `env_file: .env`, keeping API keys and secrets out of the image and out of version control.
- Dependencies — `depends_on` ensures chromadb starts before backend, and backend before frontend, matching the runtime dependency chain.

10. Container Deployment and Testing

10.1 Execution

	docker	compose	compose	up	build
	docker	compose	compose	up	-d
NAME			STATUS		PORTS
ai-interviewx-backend	Up	2 minutes		0.0.0.0:8000->8000/tcp	
ai-interviewx-frontend	Up	2 minutes		0.0.0.0:3000->3000/tcp	
ai-interviewx-chromadb	Up 2 minutes		0.0.0.0:8001->8000/tcp		

10.2 Testing Procedure and Evidence

- Health check: GET `http://localhost:8000/health` returns `{"status": "ok"}`, confirming the backend and its ChromaDB connection are live.
- Functional test: a sample resume and job description were uploaded through the frontend at `localhost:3000`; a full mock-interview session was completed end to end.
- Log verification: docker compose logs backend confirmed retrieval calls hitting ChromaDB and successful LLM responses for each question.
- Container restart test: `docker compose down && docker compose up -d` was run to confirm the ChromaDB volume persisted the vector index across restarts.

(Screenshots of the running containers, browser session, and terminal logs are to be inserted here in the submitted report.)

11. Benefits of Git and Docker

- Collaboration — feature branches let backend (RAG) and frontend (UI) work proceed in parallel without blocking each other.
- Version management — full, auditable commit history makes it possible to trace when and why any change was introduced, and to revert safely.
- Portability — the Dockerized stack runs identically on a laptop, an evaluator's machine, or a cloud VM, with no manual environment setup.
- Deployment — `docker compose up` reproduces the entire multi-service stack (API, UI, vector DB) in a single command, simplifying demos and grading.
- Scalability — the backend service can be scaled independently (e.g., multiple Uvicorn workers or replicas) as interview session volume grows.
- Maintainability — clear branch history plus isolated containers make it straightforward to upgrade one component (e.g., swap the embedding model) without destabilizing the rest of the stack.

12. Challenges and Improvements

12.1 Challenges Encountered

- Embedding dimension mismatches when swapping sentence-transformer models required a full re-index of the ChromaDB collection.
- Coordinating startup order between backend and chromadb needed `depends_on` plus an application-level retry/backoff on first connection, since `depends_on` alone does not wait for service readiness.
- Managing API keys safely across local development and containerized runs required disciplined use of `.env` and `.gitignore` to avoid accidental commits.

12.2 Suggested Improvements and Future Enhancements

- Add a CI pipeline (GitHub Actions) to run tests and build images automatically on every push to develop.
- Introduce a healthcheck: block in `docker-compose.yml` so backend waits for a genuinely ready chromadb rather than relying on `depends_on` ordering alone.

- Adopt semantic-release or a CHANGELOG generated from Conventional Commit history.
- Deploy the containerized stack publicly (e.g., Hugging Face Spaces, Render, or a small cloud VM) so the project is demoable directly from the resume link.

Appendix — Rubric Reference

Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Problem Analysis & Project Design (20%)	Clear problem, objectives, requirements, outcomes; well-organized structure with justification.	Good analysis, minor omissions.	Basic analysis, limited justification.	Incomplete/unclear.
Git Repository & Branching Strategy (20%)	Correct init, appropriate branching, clear workflow.	Mostly correct.	Basic setup, limited branching.	Incorrect/incomplete.
Version Control Workflow & Commit History (10%)	Meaningful commits, branching, merging, conflict resolution, professional messages.	Most operations correct.	Limited workflow/practices.	Poor workflow.
Docker Implementation (20%)	Well-designed Dockerfile & Compose with clear explanation.	Functional, minor issues.	Basic, limited explanation.	Incomplete/incorrect.
Deployment, Testing & Results (15%)	Deployed and tested with evidence.	Minor issues.	Partial deployment/testing.	Unsuccessful/insufficient evidence.
Documentation, Challenges & Future Enhancements (15%)	Professional report with diagrams, references, discussion.	Good, minor omissions.	Basic documentation.	Poor/insufficient.